
Deep Learning Architectures for Text Classification: Toxic Comments, A Case Study

Sebastiaan de Haas (VU: 2579828), Aleksei Kashuba (VU: 2569649), Thomas Pfann (VU: 2517946), Jannes Timm (VU: 2575272), Sam Wagenvoorde (VU: 2543989)

March 2018

Abstract

This paper delves into the hypothesis if *Character-level feature extraction can improve performance of document classifiers in informal sentiment analysis settings, such as classifying toxicity of forum comments*. This hypothesis is tested using four different Neural Networks (NN), a Character Level Convolutional NN, a Word Level Convolutional NN, a Long Short Term Model and a combinational Convolutional & Recurrent NN and compared the results with a Logistic regression and Naïve Bayes Support Vector Machine baseline. Our results are inconclusive with regards to our hypothesis, the character-level models we present do not perform better than the word-level models.

Keywords: Convolutional & Recurrent Neural Networks, Deep learning, Long Term

Short Memory

1 Introduction

The field of Natural Language Processing (NLP) has been rapidly evolving in the past years due to breakthroughs in the development of deep learning algorithms and availability of computing power & data. Specifically recurrent neural networks (RNNs) and most recently convolutional neural networks (CNNs) have been used to do sentiment classification of text quite successfully [1] [2] [3]. In this paper we explore a variety of NN architectures and their performances on the task of sentiment analysis on a specific dataset, a collection of labeled Wikipedia comments with various types of toxicity. The dataset is provided by the Toxic Comment Classification Challenge from Kaggle, a platform for predictive modeling and analytics competitions [4].

1.1 Hypothesis

Specifically the question whether character-level feature extraction can improve performance over word-level embeddings is explored. Recent research has suggested the possibility of superior performance by character-level classifiers. Santos et al (2014) demonstrated good results through the addition of one Convolutional layer (Conv-layer) to extract character features and Zhang et al (2015) achieved state-of-the-art results using character-level CNNs compared to traditional models using bag of words and deep learning models such as word-based CNNs and RNNs [5] [6]. The informal setting of our dataset, which contains many slang words and typos due to its nature of Internet forum comments, leads us to believe that a character-level approach can be successful. Considering the fact that toxic comments are often intentionally misspelled insults and contain excessive use of punctuation, a character-level classifier seems to have more potential to deduct semantics than a word-level classifier, which operates with a limited vocabulary and therefore misses many misspelled words. Thus, we intend to investigate the hypothesis:

Character-level feature extraction can improve performance of document classifiers in informal sentiment analysis settings, such as classifying toxicity of forum comments.

1.2 Approach

First we analyze the dataset and discuss the pre-processing method before establishing a baseline performance through a Logistic Regression model and a Naive Bayes Support Vector Machine (NB-SVM) Classifier, both using TF-IDF vectorization to transform the comments to bag-of-words representations. Then we demonstrate different NN architectures, based on LSTM-layers (Long-Short-Term-Memory, a specific type of recurrent layer) and Conv-layers. Since RNNs and CNNs have produced the most promising results lately, our architectures are based on these 2 types. We incorporate the character-level approach in one CNN-based model and one mixed CNN-RNN model, which are contrasted with a more traditional word-level CNN and a RNN.

2 Exploratory data analysis

The dataset has a high class imbalance; from the total amount of comments ($\approx 160k$) 89.83% is clean ($\approx 143k$). Figure 1 shows the data distribution across the different categories.

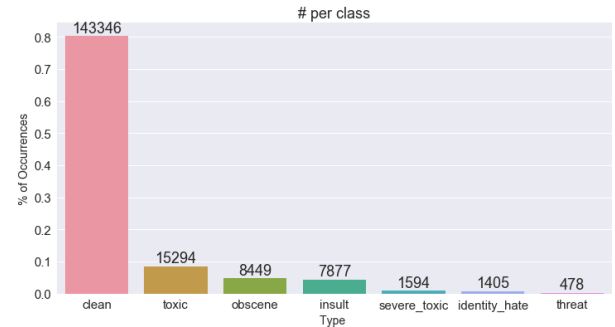


Figure 1: High class imbalance

In Figure 2 one can find the correlation between the different classes. The threat class seems to be more distinct than the others, since the highest correlation is with one of the other classes is 0.16. The insult class has a relatively high correlation the average correlation is above 0.5, so it seems it has more in common with the other classes.



Figure 2: Class correlation

Figure 3 shows a histogram of the amount of characters of the comments in the dataset. Most comments seem to be around 100 characters long, but there are also comments going up to 1500 characters. The mean length is 394.07.

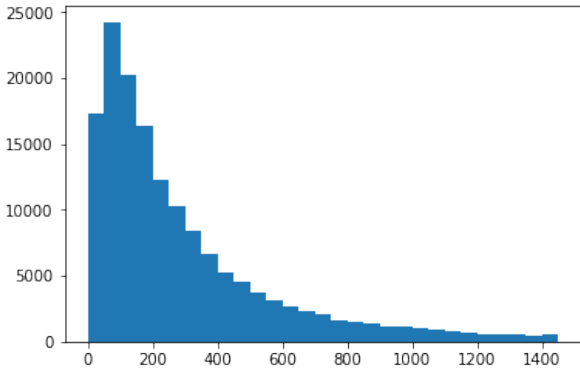


Figure 3: Histogram of comment lengths (amount of characters)

2.1 Data preprocessing

Considering the format of the comments in the dataset which sometimes include metadata, the data needs to be cleaned to prevent overfitting on those specific details. Usernames, timestamps and IP-addresses are removed from all comments. The mean length of the unmodified comments is around 400 characters, so we choose this number as a fixed length for our input sequences, which is achieved by trimming longer comments and padding shorter ones.

2.2 Splitting of training and testing data

80% of the data was used for model training, 10% for hyperparameter search and the remaining 10% for testing.

3 Exploring different baseline models

In classification problems we are interested in the conditional probability $\mathbb{P}(y|x)$, where y is the predicted class and x is the feature vector. This could be accomplished with the generative model Naive Bayes, which models the joint probability distribution $\mathbb{P}(x,y)$ and uses Bayes rule along with the assumption of independence between the features [7]. However, if one wants to avoid this assumption it is possible to estimate $\mathbb{P}(y|x)$ directly, by using Logistic regression, a so called discriminative model [8]. Both models have their perks, where discriminative learning yields a lower asymptotic error, a generative classifier can also approach its (although

higher) asymptotic error much faster [9].

3.1 Logistic Regression

This logistic regression baseline uses both words and n-grams, where n-grams are character sequences of length n . The reason why n-grams are used as well as words is that people often modify obscene words with additional characters, e.g. *'f*ckkk'*. Using character n-grams can potentially detect those. The Kernel performance can be found in Table 5 in Section 8. This yields an accuracy of 0.9792.

3.2 Naive-Bayes Support Vector Machine

The second baseline is a combination of generative (NB) and discriminative classifiers (SVM) [10]. The SVM uses bigrams of words (e.g. 2 consecutive words) in their bag-of-features representation. In a bag-of-feature model a text is represented as a multiset where multiple occurrences of in this case bigrams are allowed, by definition this is not possible in a regular set. This is useful as the bag-of-features representation gives us the ability to take into account multiple occurrences of bigrams in the corpus, therefore we are able to find multiple occurrences of insulting bigram expressions, e.g. *'drop dead'*. The SVM takes as input feature the log over the ratio between the occurrences of the bigrams in the corpus[10]. This comparison gives us the ability to compute the importance of a bigram relative to the other bigrams. The Kernel performance can be found in table 6 in Section 8. The model yields an accuracy of 0.9823.

4 Building Blocks

4.1 Embedding Layer

The embedding layer is the layer responsible for feature extraction and transforming one-hot encoded tokens (in our case we use indexes, which encapsulate the same information; a specific token out of a limited vocabulary) to dense real-valued vectors of fixed size [11]. Optimally the layer learns to encode the tokens in a way that captures semantics, specifically semantically equivalent or similar tokens are pro-

jected to vectors that are close to each other (small Euclidian distance) [12]. Embedding layers solve issues like high-dimensionality and sparsity that come with one-hot-encoded vectors [13].

4.2 Conv-layer

The intuition for using a Conv-layer is to detect local associations of features from the earlier layer and mapping the associations to a feature map [14]. In a convolutional layer a filter is applied to a window of values in the input sequence the weighted sum of which is passed through a non-linear function to produce a new feature. Such a filter is applied to all possible windows in the input to generate a feature map[3].

4.3 LSTM-layer

A long short-term memory (LSTM) network is a RNN composed of LSTM blocks. Most LSTM blocks have a cell, an input gate, an output gate and a forget gate. The cell remembers the values over arbitrary time intervals. The other three gates can be thought of as a regular artificial neuron. Intuitively, they are the regulators of the values that propagate through the connections in the LSTM [15]. Figure 4 depicts the LSTM's architecture.

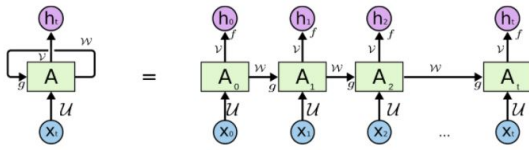


Figure 4: LSTM

4.4 Dropout

Overfitting is a proper problem in NNs. A solution can be dropout, which randomly drops nodes and the corresponding connections from the NN during training. This prevents nodes from co-adapting too much [16].

5 Models

In the following we describe 4 different NN models, based on CNNs and RNNs, operating on character and word level. We specifically

contrast CNNs and RNNs and character-level embeddings with word-level embeddings.

5.1 Character-level CNN

5.1.1 Motivation

Convolutional layers have shown to be incredibly effective in tasks involving processing of audio and images. Operating under the assumption that text can be treated as a raw signal consisting of characters, Zhang et al.[6] present an architecture consisting of nine layers with the hope that a network that is deep enough can extract long-term dependencies in the sequence. We see the greatest advantage of this network for our case in its ability to learn from misspelled words and emoticons which are prominent in informal settings.

5.1.2 Architecture overview

The model contains 6 one dimensional convolutional layers the first two of which, as well as the last one, are followed by temporal (global) max pooling to reduce the dimensions of the layers while extracting the most prominent features. Two fully connected layers with dropout were put on top of the network to perform classification.

5.1.3 Performance

The best validation accuracy we managed to achieve is 0.9663 while using both lower and upper case characters for input. The authors of the original paper achieved better performance after restricting the alphabet to only lower case characters. We hypothesize that this is due to the fact that people often express shouting in upper case. Unfortunately the size of our dataset and availability of computational power did not allow us to achieve baseline level performance with this architecture rendering this approach impractical for the task.

5.2 Word-level CNN

5.2.1 Motivation

Due to the impractical difficulty of training a fully convolutional neural network on characters, a new layer of abstraction needs to be

added. To that end an architecture that uses word embeddings as the first layer of the network has been investigated [3]. Word embeddings have proven to be effective universal feature extractors and allow for knowledge transfer by using weights that had been trained on larger datasets.

5.2.2 Architecture overview

The network consists of multiple convolutional layers with different filter sizes, each generating a feature map. Global max pooling is applied to those layers to generate input for a fully connected layer with dropout. We removed softmax, since in our task classes may overlap.

5.2.3 Performance

Overall, this architecture has shown much higher accuracy than the character based approach with validation accuracy of 0.9820. Hyperparameter search was performed for this model but in the end the hyperparameters from the original paper worked best. In this architecture feature maps can be extracted from multiple embeddings at the same time. We compared the performance of using embeddings with weights from word2vec[17], randomly initialized weights and the combination of both embeddings. Using a trainable embedding with word2vec weights showed the best performance.

5.3 LSTM with word embeddings

5.3.1 Motivation

Document classification is most prominently associated with RNNs such as LSTMs or GRUs, since they are believed to more accurately capture the context of words in sentences than other non-recurrent networks [18]. We choose to apply a relatively standard LSTM model to our sentiment analysis task.

5.3.2 Architecture Overview

The model uses an embedding layer which was described earlier. The matrix of word-vectors is fed into one bidirectional LSTM-layer, after which max-pooling is applied. We apply

dropout, to reduce overfitting, before feeding the output in two densely connected layers to perform the classification, just like in all other models.

5.3.3 Performance

After tuning hyperparameters and finding the right number of epochs to train, our model performed significantly better than the baselines. It reached around 98.36% accuracy on our validation set. It seems that this simple traditional approach easily beats our previous 2 models.

5.4 Combining Convolutional and Recurrent Layers

5.4.1 Motivation

It is possible to combine convolutional & recurrent NN's, the two mainstream architectures, in to a combined model. Such a model, with two or three convolution layers followed by a recurrent layer, achieves comparable results to a pure convolutional model while using less convolutional layers [19]. Furthermore, the model performs best when the number of classes is large, and the training size relatively small (<1500) [20].

5.4.2 Architecture Overview

Our combined CNN/RNN model is inspired by the work of Xiao et al, which is the first to attempt a mixed model on character-level [20]. Figure 5 gives a representation of the proposed network and how it differs from a pure convolutional network.

Specifically due to constraints in time and processing power we opt for the simplest architecture presented in the paper. The model combines one embedding layer with two successive convolutional layers followed by dropout. Then a single bidirectional LSTM-layer is applied, followed by another dropout layer. The result is fed into two densely connected layers for classification.

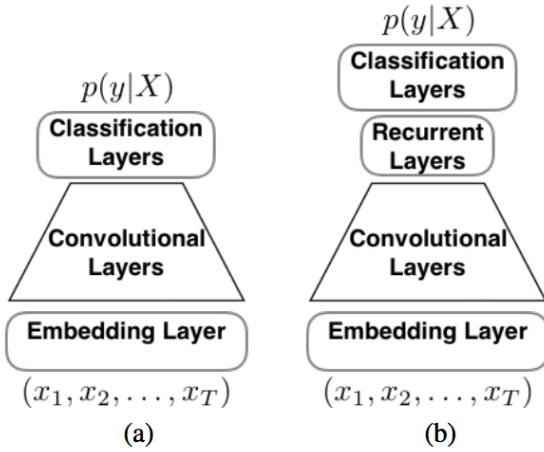


Figure 5: Illustration containing (a) a convolutional network and (b) proposed convolutional-recurrent network

5.4.3 Performance

The combined model performs worse than the word-level CNN and RNN as well as both baselines. Due to limited processing power available we could not do a very extensive hyperparameter search, so this might explain the low accuracy.

6 Results

In the following we report the accuracy on the validation set for each of the models presented earlier. The accuracy value is an average over the different class accuracies. The combinations of hyperparameters that performed best for each model are reported thereafter.

6.1 Performance details

Model	Accuracy
Log Reg Baseline	0.9792
NBSVM Baseline	0.9823
Char-level CNN	0.9633
Bag-of-words CNN	0.9820
LSTM word embeddings	0.9836
Conv&Rec NN	0.9732

Table 1: Accuracy results across different models

6.2 Hyperparameter search method

Hyperparameters in machine learning are those parameter whose values are set and not updated throughout the learning module. We used a module that is based on the Tree-structured Parzen Estimator Approach (TPE) algorithm. It models $\mathbb{P}(x|y)$ and $\mathbb{P}(y)$, here x represents hyperparameters and y the associated quality score, as opposed to modeling $\mathbb{P}(y|x)$ directly. TPE computes $\mathbb{P}(x|y)$ by a transformation on the generative process; the configuration’s prior distributions are replaced with non-parametric densities [21].

6.3 Hyperparameter values

6.3.1 Character level CNN

We did not perform hyperparameter search for this model due the difficulty of training and used the same hyperparameters as in the original paper [6].

6.3.2 Word level CNN

The word level CNN hyperparameters that we used can be found in Table 2.

Parameter	Value
Dropout	0.5
Receptive field 1	2
Receptive field 2	3
Receptive field 3	5

Table 2: Hyperparameters Word-level CNN model

6.3.3 LSTM model

We searched for values of the dropout rate of the two dropout layers and the size of the dense layer. The results can be found in Table 3.

Parameter	Value
Dense size	50
Dropout rate 1	0.56
Dropout rate 2	0.13

Table 3: Hyperparameters LSTM model

6.3.4 Convolutional & Recurrent model

We searched for values of the dropout rates of the two dropout layers, receptive field sizes of the convolutional layers, stride sizes of the convolutional layers and the size of the first dense layer after the LSTM-Layer. The following values (see Table 4) were found to produce the best performing model.

Parameter	Value
Dropout 1	0.71
Dropout 2	0.15
Receptive field 1	3
Receptive field 2	2
Stride size 1	2
Stride size 2	1
Dense	50

Table 4: Hyperparameters Convolutional Recurrent model

7 Conclusion

We created 4 models to investigate our hypothesis:

Character-level feature extraction can improve performance of document classifiers in informal sentiment analysis settings, such as classifying toxicity of forum comments.

Two Character based models (Char-level CNN, Conv&Rec NN) and 2 word based models (Word level, LSTM word embeddings). To find optimal hyperparameters we used the TPE algorithm and found optimal values for our four models. The accuracy results and our baseline can be found in 1. Only the LSTM model beats the baseline, while the word-level CNN comes close. The two character-level models perform much worse than the baseline. From the results we can not confirm the advantage of using character-level feature extraction for this specific sentiment analysis task. The character-level CNN seems to be very problematic, since the amount of layers made it impossible to do extensive training and hyperparameter tuning. The mixed Conv-Rec model seems more promising, as it is not very deep and therefore easier to train. Our hyperparameter search did not cover enough configurations to rule out the possibility that this type of architecture can achieve better accuracy than the word-level models. In order to establish this, further research would be needed in those directions. Given the results from the models we can conclude that we cannot achieve significantly better performance with either the word or character embedding based models. We therefore conclude that we can not confirm our hypothesis.

References

- [1] Siwei Lai, Liheng Xu, Kang Liu, and Jun Zhao. Recurrent convolutional neural networks for text classification. In *AAAI*, volume 333, pages 2267–2273, 2015.
- [2] Ji Young Lee and Franck Dernoncourt. Sequential short-text classification with recurrent and convolutional neural networks. *arXiv preprint arXiv:1603.03827*, 2016.
- [3] Yoon Kim. Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*, 2014.
- [4] Anthony Goldbloom. Kaggle. <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>.
- [5] Cicero dos Santos and Maira Gatti. Deep convolutional neural networks for sentiment analysis of short texts. In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 69–78, 2014.
- [6] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in neural information processing systems*, pages 649–657, 2015.
- [7] Saeed Ghahramani. *Fundamentals of probability*, volume 2. Prentice Hall New Jersey, 2000.

- [8] Xiaojin Zhu. *Advanced NLP: Text Categorization with Logistic Regression*. University of Wisconsin, 2007.
- [9] Andrew Y Ng and Michael I Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive bayes. In *Advances in neural information processing systems*, pages 841–848, 2002.
- [10] Sida I. Wang and Christopher D. Manning. Baselines and bigrams: Simple, good sentiment and topic classification. *Stanford*, 2012.
- [11] Joseph Turian, Lev Ratinov, and Yoshua Bengio. Word representations: a simple and general method for semi-supervised learning. In *Proceedings of the 48th annual meeting of the association for computational linguistics*, pages 384–394. Association for Computational Linguistics, 2010.
- [12] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel Kuksa. Natural language processing (almost) from scratch. *Journal of Machine Learning Research*, 12(Aug):2493–2537, 2011.
- [13] Rutger Ruizendaal. Why you need to start using embedding layers. *Towards Data Science*, 2017.
- [14] Can Aykin. *Layers of a Convolutional Neural Network*. Technical University of Munich (TUM), 2017.
- [15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [16] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [17] Word2vec. <https://code.google.com/archive/p/word2vec/>.
- [18] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [19] Chunting Zhou, Chonglin Sun, Zhiyuan Liu, and Francis Lau. A c-lstm neural network for text classification. *arXiv preprint arXiv:1511.08630*, 2015.
- [20] Yijun Xiao and Kyunghyun Cho. Efficient character-level document classification by combining convolution and recurrent layers. *arXiv preprint arXiv:1602.00367*, 2016.
- [21] James S Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyperparameter optimization. In *Advances in neural information processing systems*, pages 2546–2554, 2011.

8 Appendix

Class	Accuracy
toxic	0.969
severe toxic	0.988
obscene	0.984
threat	0.983
insult	0.977
identity hate	0.974

Table 5: Results of the Logistic Regression baseline

Class	Accuracy
toxic	0.9799
severe toxic	0.9842
obscene	0.9892
threat	0.9858
insult	0.9818
identity hate	0.9726

Table 6: Results of the Naive-Bayes Support Vector Machine